

Progress Report 2

Measuring the Recall Cost of Approximate Nearest Neighbor Search: A Parameter Selection Heuristic for Vector Databases

EMMANUEL C. ENENTA

Senior Seminar · Fisk University · Weeks 5 and 6

1. Project Overview

This project measures how much recall approximate nearest neighbor (ANN) indexes lose at their library default settings, and tests whether the setting a dataset needs can be predicted from a cheap statistic of the data instead of found by an exhaustive sweep. In weeks 5 and 6 I built that prediction model, a step originally planned for weeks 9 to 11, and tested it on datasets it had not seen. I also doubled the real datasets from three to six, checked result-set size and full-size collections, and put confidence intervals on the main numbers. Two of these checks changed conclusions from Progress Report 1.

1.1 How the methods work

Recall and latency. For every query, exact brute-force search gives the true 10 nearest neighbors. Recall@10 is the share of those 10 that the index actually returns, averaged over all queries: 1.0 is perfect, and 0.335 means about two of every three true neighbors were missed. Latency is the time per query on one CPU thread. Every setting studied here trades one for the other.

IVF (inverted file index). When the index is built, k-means splits the collection into n list cells (100 by default, 1,024 in my main sweeps), each with a center, and every vector is filed under its nearest center. To answer a query, the index compares it with all the centers and scans only the vectors in the n probe closest cells. A true neighbor is missed when it sits just across a cell border, in a cell that was not scanned (Figure 1, left). Raising n probe scans more cells, so recall rises and the time grows roughly in step with the number of vectors scanned. FAISS uses n probe = 1 by default. IVF-Flat keeps the full vectors; IVF-PQ compresses each one into a short code, 8 bytes at the default settings (product quantization), which saves memory but makes every distance approximate, so even scanning every cell cannot reach full recall.

HNSW (hierarchical navigable small world graph). Every vector becomes a node linked to about $M = 16$ of its near neighbors. A random few are also placed in sparser upper layers with longer links, like express lanes. A search enters at the top layer, keeps stepping to whichever neighbor is closer to the query, drops a layer when it cannot get closer, and repeats down to the bottom layer, which holds every vector (Figure 1, right). There it keeps a list of the ef best candidates found so far and keeps expanding them until none of their neighbors is closer. With a small ef the walk is fast but greedy and can settle in the wrong neighborhood; a larger ef explores more of the graph, which raises recall and costs time. `hnswlib`'s default is $ef = 10$, and it silently raises ef to k whenever k is larger.

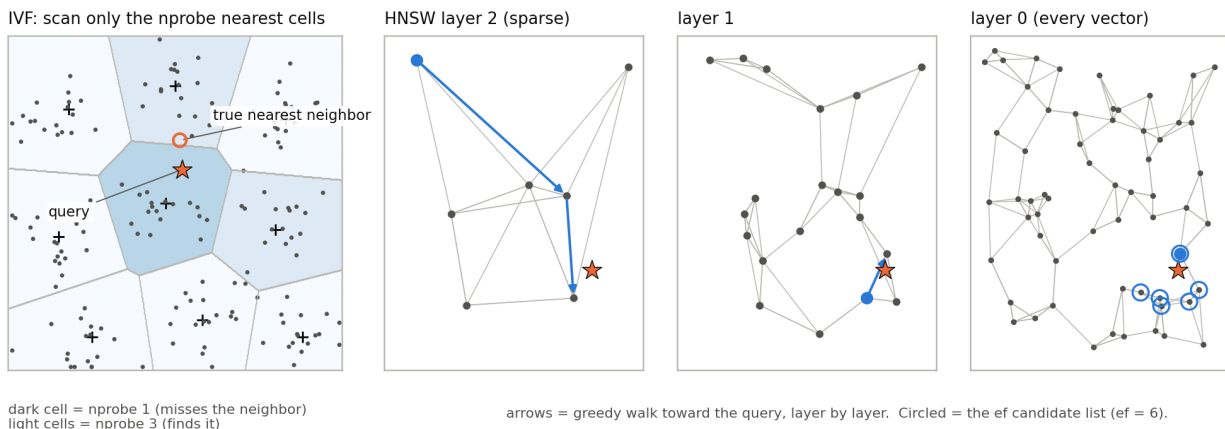


Figure 1. How each index answers a query (toy 2-D data, for illustration). Left: IVF scans only the nprobe cells nearest the query, so a neighbor just across a border is missed at nprobe = 1. Right: HNSW walks greedily down a stack of graphs, then keeps ef candidates on the bottom layer.

The trade-off in numbers. Table 1 shows both settings on SIFT. Moving HNSW from the default ef = 10 to ef = 48 lifts recall from 0.729 to 0.961 for about three times the query time. Moving IVF-Flat from nprobe = 1 to nprobe = 32 lifts it from 0.308 to 0.967 for about twelve times the time. Past that point each extra bit of recall gets expensive, so the useful question is the smallest setting that reaches a target, which this project calls the oracle setting.

index	setting	recall@10	ms per query	note
HNSW	ef = 10	0.729	0.041	library default
HNSW	ef = 48	0.961	0.119	oracle for 0.95
HNSW	ef = 1024	1.000	1.727	
IVF-Flat	nprobe = 1	0.308	0.030	library default
IVF-Flat	nprobe = 32	0.967	0.365	oracle for 0.95
IVF-Flat	nprobe = 256	1.000	2.586	

Table 1. The speed and accuracy trade-off on SIFT (200,000 vectors, IVF with 1,024 cells, one search thread).

Local intrinsic dimensionality (LID). Vectors are stored in 25 to 784 dimensions, but real data rarely uses all of them; handwritten digits vary along far fewer directions than their 784 pixels. LID estimates how many dimensions the data effectively has around one point. I compute it from the distances to that point's 20 nearest neighbors with the Levina-Bickel maximum-likelihood estimator, $LID = -1 / \text{mean}(\ln(r_i / r_{20}))$ over $i = 1$ to 19. If the 20th neighbor is much farther away than the 1st, the space is locally low-dimensional; if all 20 sit at nearly the same distance, it is high-dimensional. High LID hurts both index types for the same reason: the true neighbors are barely closer than many near misses, so they scatter over more IVF cells and the graph walk has more ways to turn the wrong way. That is why the setting needed for a fixed recall grows with LID, and why LID is the main input to my model.

2. Work Completed

The codebase grew from 889 to 1,701 lines of Python, and this period added 2,556 timed measurements, for a total of 3,213.

More data. I added Fashion-MNIST and MNIST (60,000 images, 784 dimensions) and GloVe-25 (25-dimensional word vectors), plus six synthetic datasets whose intrinsic dimension I control, from 4 to 48. The synthetic sets let me check the dimensionality estimator against known answers. It reads 4.3 for a true 4 and 7.6 for a true 8, but only 22.8 for a true 48, so it underestimates as the dimension grows.

Fine-grained sweeps. 21 HNSW ef values and 16 IVF nprobe values on all twelve datasets, three timed repeats each, to find the cheapest setting that reaches a target recall.

The prediction model. $\log_2$ of the needed setting is a linear function of one feature, the local intrinsic dimensionality (LID) measured at the query points. I tested it leave-one-dataset-out on the six real datasets: fit on everything else, predict the held-out dataset, then check the recall and latency that prediction actually gets. Baselines are the library default, a safe constant (the largest value that worked in training), a median constant and the oracle. A second version adds a margin fixed in advance.

Live demos. `scripts/predict_demo.py` generates a dataset the model has never seen, measures its LID in under a second, predicts the setting, and checks it on the spot.

Repository. Code, raw results and both demos: github.com/chinedu-2002/ann-recall-study

3. Evidence of Progress and Results

3.1 The default gap on six datasets

Across all eighteen dataset-and-index combinations the mean default recall@10 is 0.536, ranging from 0.128 to 0.930. That corrects Progress Report 1, which said no default reached 0.75: on MNIST and Fashion-MNIST the HNSW default reaches 0.929 and 0.930. HNSW defaults get worse as LID rises, down to 0.515 on GloVe-100. The IVF-Flat finding holds on every dataset: a tuned index beats the default at the same latency by between 0.171 and 0.291 recall.

dataset (LID)	HNSW default recall@10	IVF-Flat default recall@10	IVF-Flat best at same latency	free gain
MNIST (16)	0.929 $\pm$ 0.014	0.642 $\pm$ 0.032	0.817	+0.175
Fashion-MNIST (18)	0.930 $\pm$ 0.013	0.698 $\pm$ 0.032	0.870	+0.171
GloVe-25 (18)	0.763 $\pm$ 0.025	0.564 $\pm$ 0.035	0.811	+0.247
SIFT (24)	0.731 $\pm$ 0.024	0.528 $\pm$ 0.034	0.773	+0.245
GloVe-100 (43)	0.515 $\pm$ 0.038	0.501 $\pm$ 0.038	0.681	+0.180
NYTimes (53)	0.679 $\pm$ 0.044	0.482 $\pm$ 0.043	0.774	+0.291

Table 2. Library defaults at $k = 10$, sorted by LID. Intervals are 95% over 300 queries.

3.2 LID predicts the setting a dataset needs

Across all twelve datasets, the log of the smallest setting that reaches recall 0.95 tracks query-point LID with a correlation of 0.963 for HNSW and 0.959 for IVF-Flat (Figure 2). For HNSW, the ef of a dataset needs doubles for every 7.3 points of LID. Synthetic and real datasets fall on the same line.

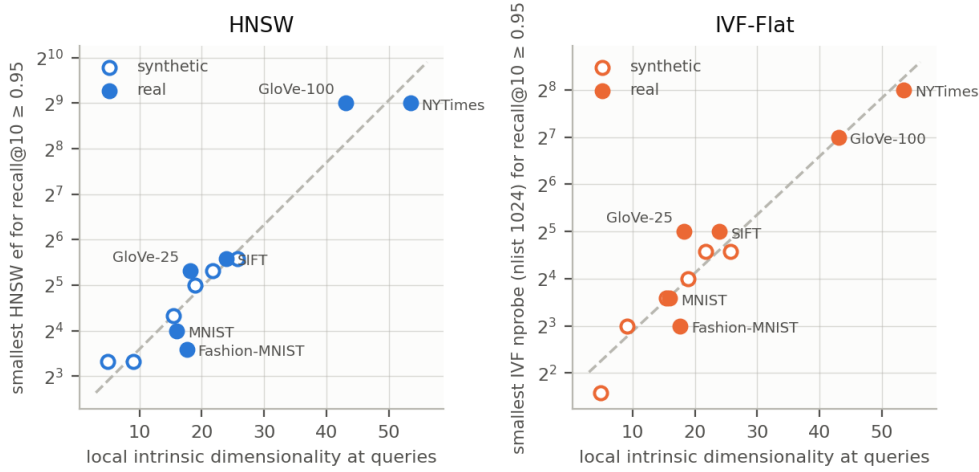


Figure 2. Smallest setting reaching recall@10 of 0.95 against LID, twelve datasets. Filled points are real, hollow points synthetic; the dashed line is the fitted model (log scale).

3.3 Testing the model on datasets it has not seen

The library default never reaches 0.95 on a held-out dataset. The safe constant works, but on HNSW it costs 4.79 times the oracle's latency. The bare model hits the target on four of six datasets at 1.17 and 1.00 times the oracle cost. With the margin it hits five of six for both index types at 1.44 and 1.42 times, about a third of what the safe constant spends. Both misses are GloVe word embeddings, which reached 0.937 and 0.938. Swapping LID for plain vector dimension makes the model barely better than guessing a constant (average error 1.68 against 1.97 log2 units, versus 0.72 with LID), so LID is what carries the signal.

method	HNSW hits	HNSW cost vs oracle	IVF-Flat hits	IVF-Flat cost vs oracle
LID model	4 / 6	1.17×	4 / 6	1.00×
LID model + margin	5 / 6	1.44×	5 / 6	1.42×
safe constant	6 / 6	4.79×	5 / 6	4.68×
median constant	2 / 6	0.54×	2 / 6	0.50×
library default	0 / 6	0.22×	0 / 6	0.10×

Table 3. Leave-one-dataset-out on the six real datasets, target recall@10 = 0.95.

3.4 Other checks

At $k = 100$, $ef = 10$ and $ef = 100$ give identical recall on all three datasets tested, to five decimals, which confirms that hnsplib silently raises ef to k . At full size the HNSW default gets worse (GloVe-100 falls from 0.515 to 0.478, SIFT to 0.696), while the IVF-Flat gain survives: a tuned index reaches 0.827 on SIFT and 0.810 on GloVe-100 at the default's latency.

4. Challenges

My headline did not survive more data. Two HNSW defaults reach 0.93, so last report's claim that no default reached 0.75 no longer holds. The IVF-Flat gap appears on all six datasets; the HNSW gap depends on the dataset.

Rebuilding the same index changes recall. NYTimes measured 0.679 last report and 0.604 this period with identical settings. Single-threaded builds reproduce 0.679 exactly, but shuffling the insertion order gives 0.643 to 0.652, and two-threaded builds give 0.590 to 0.634. That 0.089 spread is as large as the entire ± 0.044 interval from sampling queries, so last report's NYTimes figure was a favorable draw.

Size and estimator limits. The two easiest datasets are also the smallest, so part of the LID effect may be a size effect. The estimator also saturates, so the synthetic data only reaches an LID of about 26, and the high end of the model rests on two real datasets.

5. Next Steps

In weeks 7 and 8 I will separate collection size from LID with sweeps from 50,000 to 1,000,000 vectors, test a hubness feature for the GloVe misses, rebuild each oracle three times to give it an error bar, and extend the model to build settings (HNSW M, IVF nlist).